

Evaluator.py

Student worksheet – build a poker hand evaluator

Create a file called evaluator.py. Work through the steps in order. Each part adds one small piece to the final program. This worksheet does NOT assume you have implemented cards.py before.

You can treat Card as an object that has two attributes: card.rank (integer-like, e.g. 2–14) and card.suit (can be compared for equality).

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of evaluator.py, add the imports you will need.

```
from collections import Counter
from dataclasses import dataclass
from enum import IntEnum
from itertools import combinations
```

Hint: Counter helps you count duplicates such as pairs, triples, etc.

1. create the class HandCategory(IntEnum):

Inside the class:

- initialise HIGH_CARD, ONE_PAIR, TWO_PAIR, THREE_OF_A_KIND, STRAIGHT, FLUSH, FULL_HOUSE, FOUR_OF_A_KIND, STRAIGHT_FLUSH
- set them to 0, 1, 2, ..., 8 respectively

```
class HandCategory(IntEnum):
    HIGH_CARD = 0
    # continue here
```

Checkpoint:

```
print(HandCategory.FLUSH > HandCategory.STRAIGHT) # expected: True
```

2. create the class HandRank:

- use @dataclass(frozen=True, order=True) above the class
- add attributes: category and tiebreakers

- create a `@property` called `label`
- return a readable version of the category

```
@dataclass(frozen=True, order=True)
class HandRank:
    category: HandCategory
    ...
```

Hint: use `.name.replace("_", " ").title()`

3. create the class `HandEvaluator`:

- create method `best_rank(self, cards)`
- raise `ValueError` if fewer than 5 cards
- generate all 5-card combinations
- return the best result using `max()` (*refer to problem #4*)

```
class HandEvaluator:
    def best_rank(self, cards) -> HandRank:
        if len(cards) < 5:
            ...
```

Hint: Use `combinations(cards, 5)` to create every possible 5-card hand from the cards you are given

4. create the method `_rank_five(self, cards)` (inside class `HandEvaluator`):

This helper method evaluates exactly five cards. Start with the values you need for the later checks.

- extract ranks and sort descending
- count with `Counter`
- group by (count, rank)
- check flush
- check straight
- *implement ranking logic in problem #6*

```
def _rank_five(self, cards) -> HandRank:
    ...
```

Small reminder: Grouped counts help you detect pairs, triples, full houses, and four-of-a-kind hands.

5. create the method `_straight_high(self, ranks)` (in class `HandEvaluator`):

Return the highest card of a straight, or `None` if there is no straight.

- remove duplicate ranks before checking
- handle the special case A-2-3-4-5
- check windows of 5 consecutive numbers

```
def _straight_high(self, ranks: list[int]) -> int | None:
    ...
```

Hint: Duplicate ranks should not break a straight check.

6. implement ranking logic (from problem #4):

Inside `_rank_five`, check in this order:

- straight flush → four of a kind → full house → flush → straight
- three of a kind → two pair → one pair → high card

```
if is_flush and straight_high:
    return HandRank(HandCategory.STRAIGHT_FLUSH, (straight_high,))

if groups[0][0] == 4:
    ...

return HandRank(HandCategory.HIGH_CARD, tuple(ranks))
```

Hint: The tiebreakers are important. The strongest deciding value should always come first

7. final test

At the bottom of the file, or in a separate test file, try this:

```
evaluator = HandEvaluator()
# create some test cards
# print(evaluator.best_rank(cards))
```

Note: Write one small test case for each hand category to verify your logic.

Common mistakes to check

- Forgetting to sort ranks before processing.
- Incorrect use of Counter.
- Not handling duplicate ranks when checking for a straight.
- Placing tiebreakers in the wrong order.